本节内容：

- 运算符
- 判断
- 循环
- 函数

一、运算符

1.1 比较运算符

- `>` 大于
- `<` 小于
- `>=` 大于等于
- `<=` 小于等于
- `==` 左右两边的值是否相等
- `===` 左右两边是否类型和值都相等
- `!==` 左右两边是否不全等

> 开发中判断是否相等，强烈建议使用 `===`

1.2 逻辑运算符

符号	名称	日常读法	特点	口诀
&&	逻辑与	并且	符号两边都为true 结果才为true	一假则假
‖	逻辑或	或者	符号两边有一个 true就为true	一真则真
!	逻辑非	取反	true变false false变true	真变假，假变真

二、条件判断

三个关键词汇：`if`，`else`，`else if`

语法结构：

```
if(条件){
    执行语句
}else if(条件){
    执行语句
}else{
    执行语句
}
```

```
<script>
    let a = 30
    if(a == 30){
        console.log('条件成立 执行下面语句');
    }else{
        console.log('条件不成立 执行else的语句');
    }
</script>
```

三、循环

3.1 while

介绍： 根据条件循环代码块

语法：

```
while (条件){
    内部代码块
}
```

示例：

```
// 打印三次"hello world"
let i = 1
while (i <= 3){
    console.log('hello world')
    i++  // i 加一
}
```

3.2 for

介绍： 重复执行代码

好处： 把声明起始值、循环条件、变化值写到一起，让人一目了然，它是最常使用的循环形式

语法：

```
for (变量起始值; 终止条件; 变量变化量) {
    // 循环体
}
```

示例：

1. 打印三次"hello world"

```
for (let i = 1; i <= 3; i++) {
    console.log('hello world');
}
```

2. 遍历数组（**重点**）

```
let ls = ['a', 'b', 'c', 'd']

for (let i = 0; i < ls.length; i++) {
    console.log(ls[i]);
}
```

3.3 break关键字

介绍： 只能在循环中使用

作用： 当程序执行到break时会立刻杀死当前循环

3.4 continue关键字

介绍： 只能在循环中使用

作用： 当程序执行到continue时会跳过本次循环剩余的代码，直接进入到下一次循环

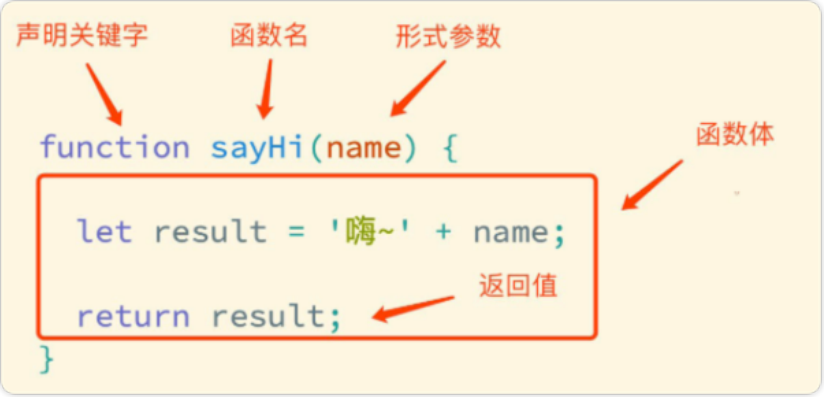
四、函数

4.1 系统函数

`console.log()、parseInt()、parseFloat()...`

4.2 自定义函数

定义语法：



- **声明关键字：** 必须使用 `function`
- **函数名：** 可以自定义名字，但需要注意名次不能以数字开头，只能由数字、字母、下划线组成。
- **形式参数：** 可以自定义名字，且形式参数的数量也没有固定与实参的数量匹配上就行，但需要注意名次不能以数字开头，只能由数字、字母、下划线组成。
- **函数体：** 可以写任何的JavaScript代码。
- **返回值：** 会把 `return` 后面的值返回给调用者

调用语法： `函数名(实参)`

示例：

```
function run(name) {
    console.log(name);
}

run('张三')
run('李四')
```

4.3 匿名函数

没有名字的函数, 无法直接使用。

使用方式：

- 函数表达式
- 立即执行函数

语法：

```
let fn = function () {
    // 函数体
}
```

调用：

```
fn()  // 函数名()
```

4.4 箭头函数

对匿名函数的简写

```
let fn = function(a){
    // 函数体
}

// 通过箭头函数可简化为
let fn = (a)=>{  // 如果形参只有一个，那么小括号也可以省略
    // 函数体
}
```

4.5 高阶函数

高阶函数可以接收另一个函数作为参数。

4.5.1 `filter()` -数组过滤

- 筛选出满足条件的元素（return后的`表达式为true`时，元素会被保留）
- 返回一个新数组
- 应用场景：数据筛选、条件过滤

```
const numbers = [1, 2, 3, 4];
const evens = numbers.filter(function (num) {
  return num % 2 === 0;
});
console.log(evens);
// 结果: [2, 4]
```

4.5.2 `forEach()` -数组遍历

- 对每个元素执行操作
- 没有返回值
- 应用场景：遍历每个元素做统一处理

```
const numbers = [1, 2, 3, 4];
numbers.forEach(function (num) {
  console.log(num);
});
// 输出: 1, 2, 3, 4
```